



**Université des sciences et de la Technologie
Houari Boumediene
USTHB – Alger**

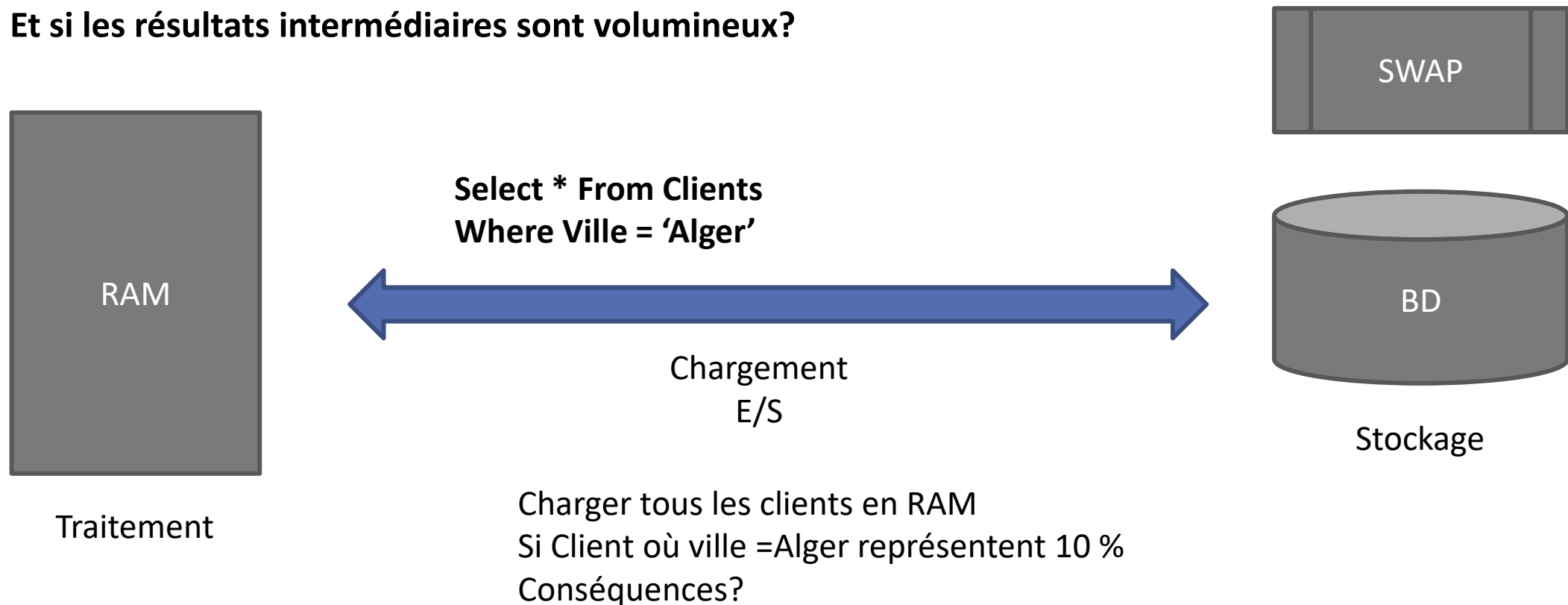
Faculté d'Informatique

ADMINISTRATION ET TUNING DE BASES DE DONNÉES

OPTIMISATION DES REQUÊTES

INTRODUCTION

- ☐ Ce que veut dire optimisation?
- ☐ Qu'est-ce qu'on optimise?
- ☐ Pourquoi il y a un espoir d'optimisation des requêtes?
- ☐ Quelles sont les sources du temps supplémentaire?
- ☐ Et si les Tables sont volumineuses?
- ☐ Et si les résultats intermédiaires sont volumineux?



ÉVALUATION DES REQUÊTES: SURVOL

☐ Évaluation d'une requête SQL:

- ☐ Analysée syntaxiquement, ensuite transformée en plan d'évaluation.
- ☐ **Plan d'évaluation:** Arbre d'opérations de l'algèbre relationnelle avec un choix d'algorithme pour chaque opération.

☐ Deux problématiques importantes dans l'optimisation:

- ☐ Énumération des plans alternatifs pour une requête
- ☐ Estimation des coûts de ces plans et choix de celui estimé être le moins cher

☐ On doit se montrer pragmatique:

- ☐ **Idéalement:** Trouver le meilleur plan.
- ☐ **Pratiquement:** Éviter les pires plans!

ARBRE ALGÉBRIQUE

- ❑ **Algèbre relationnelle** : modèle formel permettant d'exprimer et de calculer les **requêtes** sur les **relations**.
- ❑ Constituée d'un ensemble **d'opérateurs algébriques** : la sélection, la jointure, la projection, les opérateurs ensemblistes.
- ❑ Le résultat de l'exécution d'un opérateur est **toujours une relation**, ce qui permet la **composition**.
- ❑ Utilisée également pour **l'optimisation de requêtes**.
- ❑ Une requête algébrique peut se présenter sous forme d'un **arbre algébrique**.

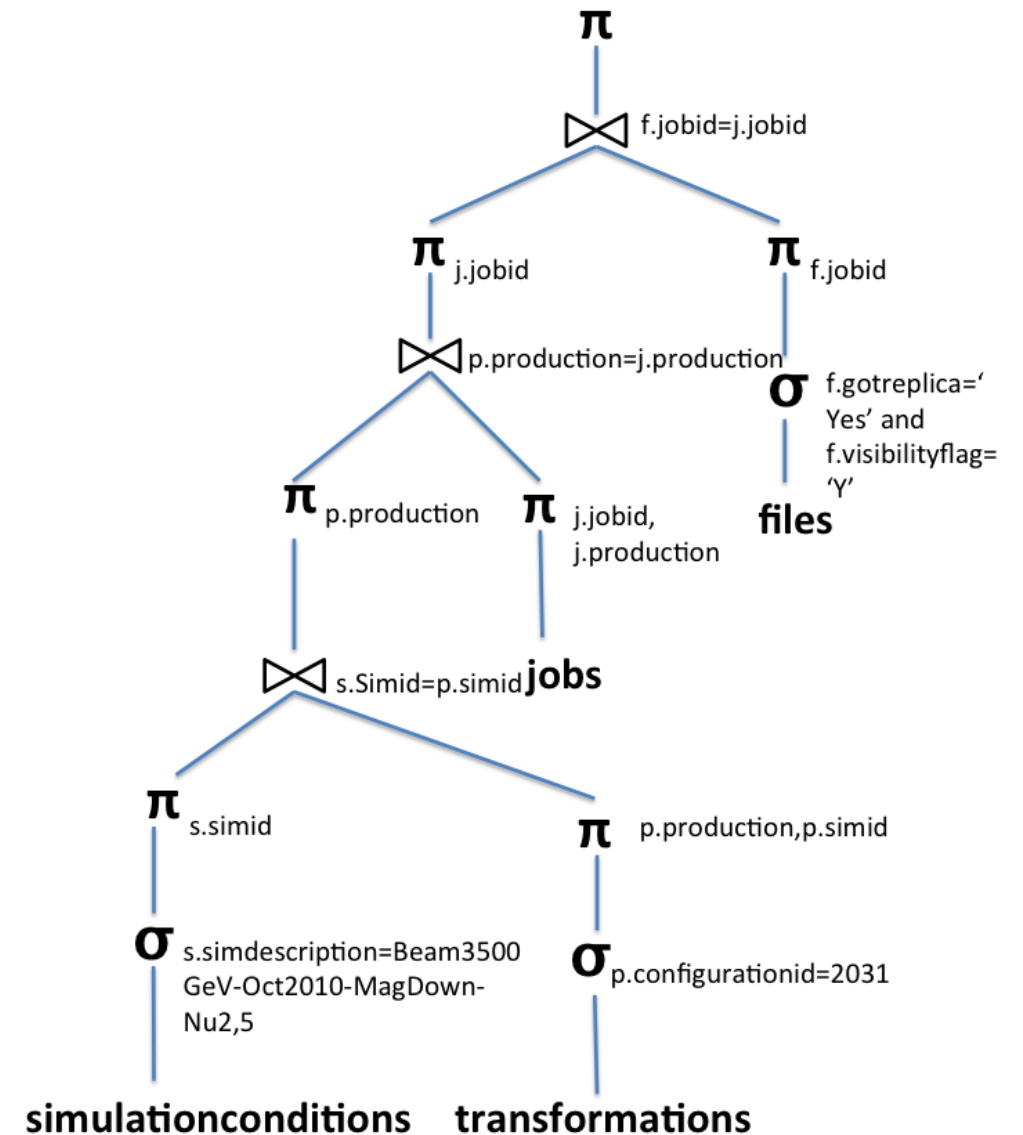
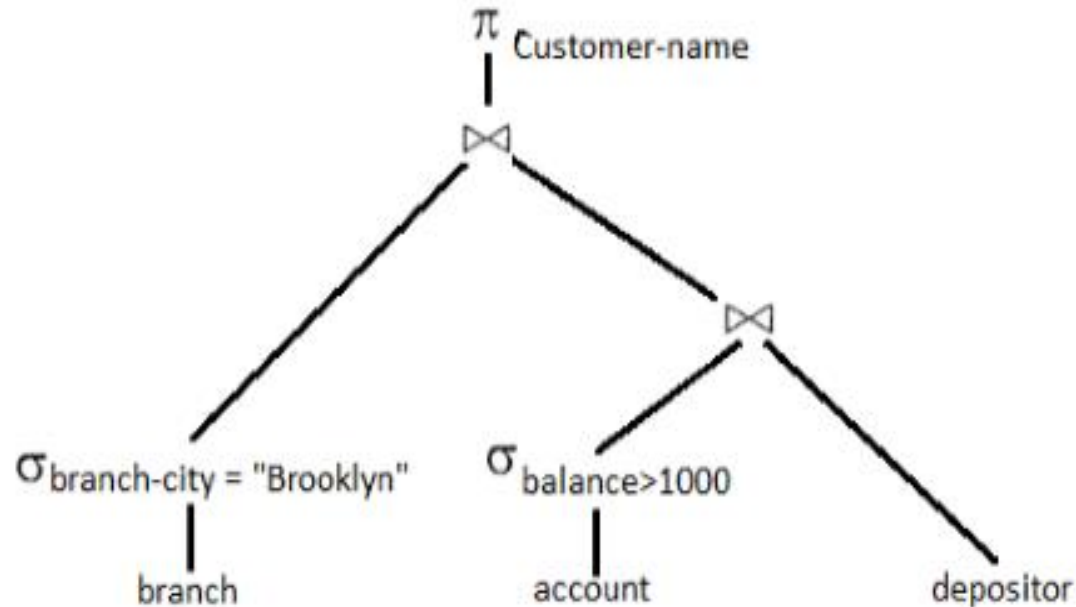
Arbre algébrique

- ❑ visualisation d'une requête sous une forme graphique d'arbre plus facile à lire qu'une forme linéaire

ARBRE ALGÉBRIQUE

Un arbre algébrique est un arbre représentant une requête

- ❑ Les **nœuds feuilles** sont les **relations de base**
- ❑ Les **nœuds intermédiaires** sont les **opérateurs**
- ❑ Le nœud **racine** est le **résultat**
- ❑ L'**arc** est un **flux** de données



STATISTIQUES ET CATALOGUES

L'évaluateur a besoin d'informations sur les relations ainsi que les indexes impliquées. Les *Catalogues* contiennent:

- ❑ pour chaque relation: # tuples (NTuples) et # pages (NPages)
- ❑ pour chaque index: # valeurs de clés distinctes (NKeys) et # pages (NPages)
- ❑ pour chaque index à arbre: Hauteur de l'Index (Height), plus petites / plus grande valeurs de clé (Low/High)

Catalogues rafraîchis périodiquement.

- ❑ Automatiquement, Manuellement

D'autres types d'infos détaillées (p.ex., histogrammes des valeurs dans certains champs) sont aussi stockées.

- ❑ **Histogramme:** structure donnant une approximation de la distribution des valeurs des données

CHEMINS D'ACCÈS ET CORRESPONDANCE D'INDEX

- ❖ Un chemin d'accès est une méthode pour puiser les tuples:
 - Utilisation du **Scannage du fichier**, ou d'un **index** qui correspond à ("**match**") une sélection (WHERE) de la requête.
- ❖ Un index à arbre correspond à une conjonction de termes qui mentionnent seulement des attributs d'un *préfixe* de la clé de recherche.
 - Index b-arbre sur a correspond à la sélection $a=5$, ou $a>5$
 - L'index à arbre sur $\langle a, b, c \rangle$ correspond à la sélection $a=5 \text{ AND } b=3$ et à $a=5 \text{ AND } b>6$, mais pas à $b=3$.
- ❖ Un index à hachage correspond à une conjonction de termes qui a un terme de la forme **attribut = valeur** pour chaque attribut de la clé de recherche de l'index.
 - L'index à hachage sur $\langle a, b, c \rangle$ correspond à $a=5 \text{ AND } b=3 \text{ AND } c=5$; mais pas à $b=3$, ni à $a=5 \text{ AND } b=3$, ni à $a>5 \text{ AND } b=3 \text{ AND } c=5$.
- ❖ Index Bitmap

TECHNIQUES D'OPTIMISATION

❑ Expressions Algébriques

- ❑ Transformation des requêtes pour obtenir la meilleure séquence d'opérations
- ❑ RBO : Rules Based Optimisation

❑ Opérations Algébriques

- ❑ Coût des différentes stratégies possibles en fonction des caractéristiques des fichiers sur lesquels sont implantées les relations
- ❑ CBO : Cost Based Optimisation

Généralement les SGBDs commerciaux combinent les deux stratégies

- ❑ Oracle utilise actuellement CBO seulement

RULES BASED OPTIMISATION

Requête (SQL)



Traduction

Requête en algèbre

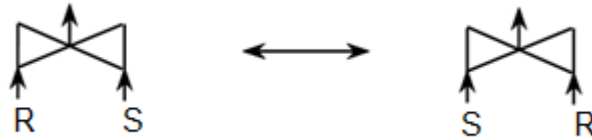


Transformation en fonction des
propriétés des opérateurs

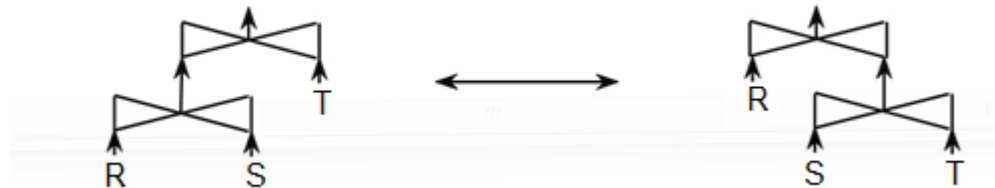
Plan optimal

PROPRIÉTÉS DES OPÉRATEURS

❑ Commutativité pour jointure et produit



❑ Associativité pour jointure et produit



❑ Il existe **N!** arbres de jointure de N relations.

PROPRIÉTÉS DES OPÉRATIONS (SUITE)

❑ Groupage des projections

$$\square \quad \pi_{A1, \dots, An} (\pi_{B1, \dots, Bm}(E)) = \pi_{A1, \dots, An} (E) \text{ --- } \{A1, \dots, An\} \text{ inclus dans } \{B1, \dots, Bm\}$$

❑ Groupage des sélection

$$\square \quad \sigma_{F1} (\sigma_{F2} (E)) = \sigma_{F1 \wedge F2}(E)$$

$$\square \text{Exemple : } \sigma_{\text{age} > 20} (\sigma_{\text{genre} = 'F'} (\text{Étudiant})) = \sigma_{(\text{age} > 20) \wedge (\text{genre} = 'F')}(\text{Étudiant})$$

❑ Inversion σ et π

$$\square \quad \pi_{A1, \dots, An} (\sigma_F(E)) = \sigma_F(\pi_{A1, \dots, An}(E)) \text{ si } F \text{ porte sur } A1, \dots, An$$

$$\square \quad \pi_{A1, \dots, An} (\sigma_F(E)) = \pi_{A1, \dots, An} (\sigma_F(\pi_{A1, \dots, An, B1, \dots, Bm}(E))) \text{ si } F \text{ porte sur } B1, \dots, Bm \text{ qui ne sont pas parmi } A1, \dots, An$$

PROPRIÉTÉS DES OPÉRATIONS (SUITE)

Inversion Sélection-Produit

- ❑ $\sigma_F (E1 * E2) = \sigma_F (E1) * E2$ si F ne porte que sur les attributs de E1
- ❑ $\sigma_F (E1 * E2) = \sigma_{F1} (E1) * \sigma_{F2} (E2)$ Si $F = F1 \wedge F2$ et F_i ne porte que sur les attributs de E_i
- ❑ $\sigma_F (E1 * E2) = \sigma_{F2} (\sigma_{F1} (E1) * E2)$ si $F1$ porte sur les attributs de $E1$ et $F2$ sur ceux de $E1$ et $E2$
- ❑ Exemple : $\sigma_{\text{Genre}='F'}(\text{Étudiant} * \text{Cours}) = \sigma_{\text{Genre}='F'}(\text{Étudiant}) * \text{Cours}$

Inversion Sélection-Union

- ❑ $\sigma_F (E1 \cup E2) = \sigma_F(E1) \cup \sigma_F(E2)$

Inversion Sélection-Différence

- ❑ $\sigma_F (E1 - E2) = \sigma_F(E1) - \sigma_F(E2)$

Inversion Projection-Produit

Inversion Projection-Union

PRINCIPES D'OPTIMISATION DES EXPRESSIONS ALGÈBRIQUES

1. Exécuter les sélections aussitôt que possible
2. Réduire la taille des relations à manipuler
 - Combiner les sélections avec un produit cartésien pour aboutir à une jointure
3. Combiner des séquences d'opérations unaires (σ, Π)
4. Mémoriser les sous-expression commune

EXÉCUTER LES SÉLECTIONS AUSSITÔT QUE POSSIBLE

Appliquer ces règles (remplacer membre gauche par droit)

1. $\sigma_F (E1 \cup E2) = \sigma_F(E1) \cup \sigma_F(E2)$
2. $\sigma_F (E1 - E2) = \sigma_F(E1) - \sigma_F(E2)$
3. $\sigma_F (E1 \text{ OP } E2) = \sigma_F (E1) \text{ OP } E2$ si F ne porte que sur les attributs de E
4. $\sigma_F (E1 \text{ OP } E2) = \sigma_{F1} (E1) \text{ OP } \sigma_{F2} (E2)$ si $F = F1 \wedge F2$ et F_i ne porte que sur les attributs de E_i
5. $\sigma_F (E1 \text{ OP } E2) = \sigma_{F2} (\sigma_{F1} (E1) \text{ OP } E2)$ si $F1$ porte sur les attributs de $E1$ et $F2$ sur ceux de $E1$ et $E2$

OP = * ou $\bowtie$

COMBINER LES SÉLECTION AVEC UN PRODUIT CARTÉSIEN POUR ABOUTIR À UNE JOINTURE

❑ Transformation si possible du produit cartésien en jointure:

❑ Si $R * S$ est l'argument d'une sélection σ_F alors si F est une comparaison entre attributs de R et de S , transformer $R * S$ en jointure : $\sigma_F (R * S) = (R \bowtie_F) S$

❑ Attention : si F est une expression ne portant que sur les attributs de R :

❑ $\sigma_F (R * S) = \sigma_F (R) * S$

SOUS-EXPRESSIONS COMMUNES À UNE EXPRESSION

- ❑ Si une expression contient plusieurs fois la même sous-expression
- ❑ Si cette sous-expression peut être lue en **moins de temps** qu'il faut pour la calculer
- ❑ Matérialiser la sous-expression commune
- ❑ **Principe de vues matérialisées**

UN ALGORITHME D'OPTIMISATION

1. Séparer chaque sélection $\sigma_{F_1 \wedge \dots \wedge F_n}(E)$ en une cascade $\sigma_{F_1} (\sigma_{F_1} \dots (\sigma_{F_n}(E)))$
2. Descendre chaque sélection le plus bas possible dans l'arbre algébrique
3. Descendre chaque projection aussi bas possible dans l'arbre algébrique
4. Combiner des cascades de sélection et de projection dans une sélection seule, une projection seule, ou une sélection suivie par une projection
5. Regrouper les nœuds intérieurs de l'arbre autour des opérateurs binaires ($\bowtie$, $*$, $-$, $\cup$). Chaque opérateur binaire crée un groupe
6. Produire un programme comportant une étape pour chaque groupe, à évaluer dans n'importe quel ordre mais tant qu'aucun groupe ne soit évalué avant ses groupes descendants.

QUELQUES PROBLÈMES

- ❑ La restructuration d'un arbre sous-entend souvent la permutation des opérateurs binaires
 - ❑ Cette optimisation ignore la taille de chaque table, les facteurs de sélectivité, etc.
 - ❑ Aucune estimation de résultats intermédiaires

❑ Exemple:

(Ouvrier ⋈ Usine) ⋈ Contrat

Si cardinalité (Usine) $\ll$ Cardinalité(Contrat) $\ll$ Cardinalité(Ouvrier) alors l'expression suivante a des chances d'être plus performante:

(Usine ⋈ Contrat) ⋈ Ouvrier

→ Optimisation basée sur les modèles de coût

MÉTHODES D'OPTIMISATION BASÉES SUR LES MODÈLES DE COÛTS

NOTIONS UTILES

- ❑ Les tables relationnelles sont stockées physiquement dans des fichiers sur disque
- ❑ Lorsqu'on veut accéder aux données, on doit transférer le contenu pertinent des fichiers dans la mémoire
 - ❑ **Fichier** = séquence de tuples
 - ❑ **Bloc (page)** = unité de transfert de données entre disque et mémoire
 - ❑ **Facteur de blocage** = nombre de tuples d'une relation qui «tiennent» dans un bloc
 - ❑ **Coût de lecture (ou écriture) d'une relation** = nombre de blocs à lire du disque vers la mémoire
(ou à écrire de la mémoire vers le disque)

ESTIMATION DU COÛT DES OPÉRATIONS PHYSIQUES

- ❑ **TempsES** : Temps d'accès à mémoire secondaire (MS)
- ❑ **TempsUCT** : Souvent négligeable
- ❑ **TailleMC** : espace mémoire centrale
- ❑ **TailleMS** : espace mémoire secondaire

MODÈLE DU COÛT D'UNE ENTRÉE- SORTIE EN MÉMOIRE SECONDAIRE

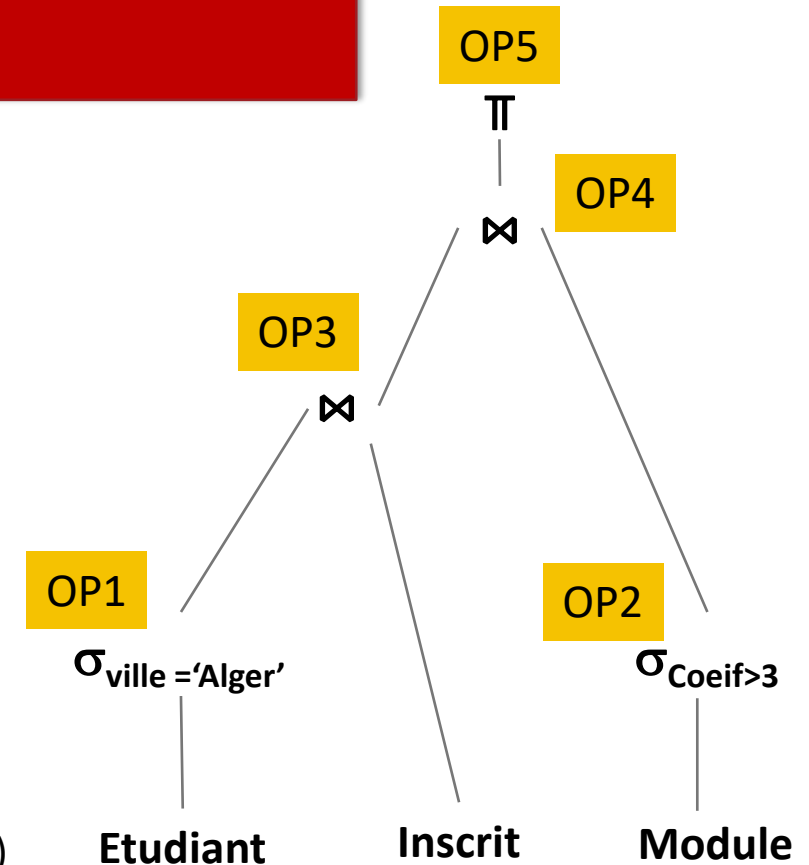
Paramètre	Signification
TempsESDisque(n)	Temps total de transfert (lecture ou écriture) de n octets du disque
TempsTrans(n)	Temps de transfert des n octets sans repositionnement
TempsPosDébut	Temps de positionnement au premier octet à transférer (ex : 10 ms)
TempsRotation	Délai de rotation (ex : 4 ms)
TempsDépBras	Temps de déplacement du bras (ex : 6 ms)
TauxTransVrac	Taux de transfert en vrac (ex : 2MB/sec)
TempsESBloc	Temps de transfert d'un bloc (ex : 11 ms)
TempsTrans	Temps de transfert d'un bloc sans repositionnement (ex : 1 ms)
TailleBloc	Taille d'un bloc (ex : 2K octets)

STATISTIQUES AU SUJET DES TABLES

Statistique	Signification
$ T $	Nombre de lignes de la table T
TailleLigne	La taille d'une ligne de la table T
FBT	Facteur de blocage moyen de T
NDIST (T,a)	Nombre de valeurs distinctes de la colonne a pour la table T
MinT(colonne)	Valeur minimum de la colonne de T
MaxT(colonne)	Valeur maximum de la colonne de T
$ T $	Nombre de pages de la table T

COÛT D'UNE REQUÊTE

- ❑ Soit R une requête composée de n opérations algébriques (OPi)
- ❑ $\text{Coût}(R) = \sum \text{Coût}(OP_i)$
- ❑ $\text{Coût}(OP_i) = \text{Nombre de pages transférées de la mémoire secondaire vers la RAM ou de la RAM vers la mémoire SWAP}$
- ❑ Pour calculer le $\text{Coût}(OP_{i+1})$
 - ❑ Il faut calculer le nombre de tuples du résultat intermédiaire de OPi ($||OP_i||$)
 - ❑ En utilisant le facteur de sélectivité de OPi : $||OP_i|| = FS * ||OP_{i-1}||$
 - ❑ Le transformer en nombre de pages en utilisant les tailles du tuples (T_Tuples) et de la page système $|OP_i| = T_tuples * ||OP_i|| / PS$
 - ❑ $\text{Taille tuple} = \sum \text{taille attributs du tuple}$
 - ❑ Une autre Formule $|OP_i| = ||OP_i|| / \#tuples \text{ par page}$
- ❑ Si RAM non suffisante : Ajouter les pages transférées en mémoire SWAP



- $T_Tuple(\sigma(T)) = T_Tuple(T)$
- $T_Tuple(T1 \bowtie T2) = T_Tuple(T1) + T_Tuple(T2)$
- $T_Tuple(\pi_{A1, A2, \dots}(T)) = \sum \text{Taille}(Ai)$

FACTEUR DE SÉLECTIVITÉ (FS)

❑ Désigne le pourcentage de lignes sélectionnées sur le nombre total de tuples

❑ Entre 0 et 1

❑ Opération trop sélective : **SF tend vers zéro**

❑ Opération non sélective : **SF tend vers 1**

❑ Exemple :

- Si étudiants habitant Alger représentent **20 %**
- Le facteur de sélectivité du prédicat **Ville='Alger'** est de **0,2%**
- Si la table Etudiant contient **100 000 étudiants**, la requête `Select * from Etudiant where Ville = Alger`
- Retournera **20 000 tuples**

FACTEUR DE SÉLECTIVITÉ

$$\square ||(\sigma(R))|| = SF * ||R||$$

- $\square SF(A = \text{valeur}) = 1 / NDIST(A)$
- $\square SF(A > \text{valeur}) = (\max(A) - \text{valeur}) / (\max(A) - \min(A))$
- $\square SF(A < \text{valeur}) = (\text{valeur} - \min(A)) / (\max(A) - \min(A))$
- $\square SF(A \text{ IN liste valeurs}) = (1/NDIST(A)) * CARD(\text{liste valeurs})$
- $\square SF(P \text{ et } Q) = SF(P) * SF(Q)$
- $\square SF(P \text{ ou } Q) = SF(P) + SF(Q) - SF(P) * SF(Q)$
- $\square SF(\text{not } P) = 1 - SF(P)$

Le coût dépend de l'algorithme (index, hachage ou balayage).

FACTEUR DE SÉLECTIVITÉ

❑ La taille d'une jointure est estimée par la formule suivante:

$$\boxed{\text{||R1} \bowtie \text{R2||} = \text{Sel} * \text{||R1||} * \text{||R2||}}$$

❑ sel: la sélectivité de la jointure

❑ Produit cartésien : **Sel=1**

❑ Jointure sur clé étrangère sur R1 : **||R1 $\bowtie$ R2|| = ||R1||**

❑ Une estimation de Sel si jointure quelconque

$$\boxed{\text{SF (R1} \bowtie_b \text{R2)} = 1 / \text{Max(NDIST(a), NDIST(b))}}$$

EXEMPLES

SELECT *
FROM R1, R2

SF = 1

SELECT *
FROM R1
WHERE A = valeur

SF = $1/\text{NDIST}(A)$ avec un modèle uniforme

ALGORITHMES GÉNÉRAUX POUR LES OPÉRATEURS RELATIONNELS

LA SÉLECTION

❑ Sélection sans index

SELECT NSS, Nom FROM Ouvrier WHERE Salaire > 15000

❑ Plan d'exécution

- Pour chaque tuple t de ouvrier faire -- accès séquentiel
- si $t.salaire > 15000$ alors **réponse** = **réponse** $\cup \pi_{NSS, Nom} (Ouvrier)$

❑ Coût = $|T| = |Ouvrier|$

❑ Sélection avec Index

- ❑ Charger l'index : Coût = $|Index|$
- ❑ Parcourir l'index en RAM , chercher les tuples à sélectionner: Coût = 0
- ❑ Charger les tuples sélectionnés : Coût = $FS * |T| = FS(Salaire > 15000) * |Ouvrier|$
- ❑ Total = $|Index| + FS * |T|$

En cas d'absence de statistiques, les estimateurs considèrent une distribution uniforme

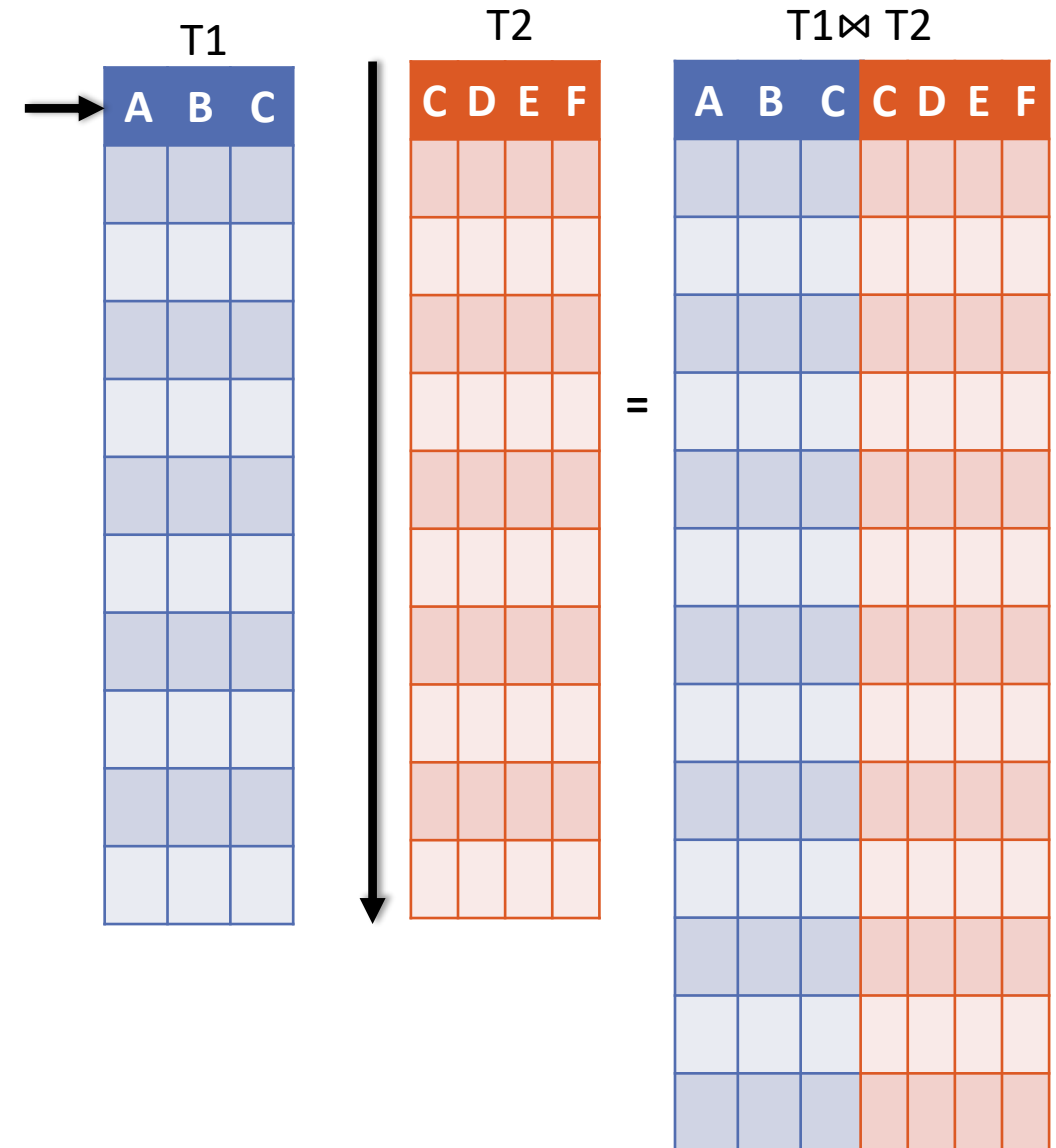
ALGORITHMES DE JOINTURE

☐ Jointure sans index

- ☐ Jointure par boucles imbriquées (nested loop)
- ☐ Jointure par tri-fusion (sort-join)
- ☐ Jointure par hachage (hash-join)

☐ Jointure avec index

- ☐ Jointure avec boucles indexées



JOINTURE PAR BOUCLES IMBRIQUÉES (JBI)

❑ $R \bowtie_F S$

❑ Principe:

- ❑ La première table est lue séquentiellement et de préférence stockée entièrement en mémoire
- ❑ Pour chaque tuple de R, il y a une comparaison avec les tuples de S

POUR chaque ligne IR de R

 POUR chaque ligne IS de S

 SI θ sur IR et IS est satisfait

 Produire la ligne concaténée à partir de IR et IS

 FINSI

 FINPOUR

FINPOUR

+ Algorithme simple

Complexité: $(|R| + (|R| * |S|))$

Si la table extérieure tient en mémoire: une seule lecture des deux tables suffit.

JOINTURE PAR TRI FUSION

❑ Plus efficace que les boucles imbriquées pour les grosses tables

❑ Principe

- ❑ Trier les deux tables sur les colonnes de jointure
- ❑ Effectuer la fusion
- ❑ Complexité: Le tri qui coûte cher

❑ Très efficace quand une des deux tables est petite (1, 2, 3 fois la taille de la mémoire)

❑ Algorithme en option dans Oracle

Coût =

- $|T1| + |T2|$ si $T1, T2$ déjà triées
- $2 * |T1| * \log_b |T1| + |T2| + |T1|$ Si $|T2|$ triée
- $2 * |T2| * \log_b |T2| + |T2| + |T1|$ Si $|T1|$ triée
- $2 * |T1| * \log_b |T1| + 2 * |T2| * \log_b |T2| + |T2| + |T1|$ si aucune table n'est triée
- b : nombre de pages du tampon en mémoire centrale disponible pour le tri

JOINTURE PAR TRI FUSION

Trier R et S par tri externe et réécrire dans des fichiers temporaires

Lire groupe de lignes GR(cR) de R pour la première valeur cR de clé de jointure

Lire groupe de lignes GS(cS) de S pour la première valeur cS de clé de jointure

TANT QUE il reste des lignes de R et S à traiter

SI $cR = cS$

Produire les lignes concaténées pour chacune des combinaisons de lignes de

GR(cR) et GS(cS);

Lire les groupes suivants GR(cR) de R et GS(cS) de S;

SINON

SI $cR < cS$

Lire le groupe suivant GR(cR) de R

SINON

SI $cR > cS$

Lire le groupe GS(cS) suivant dans S

FINSI

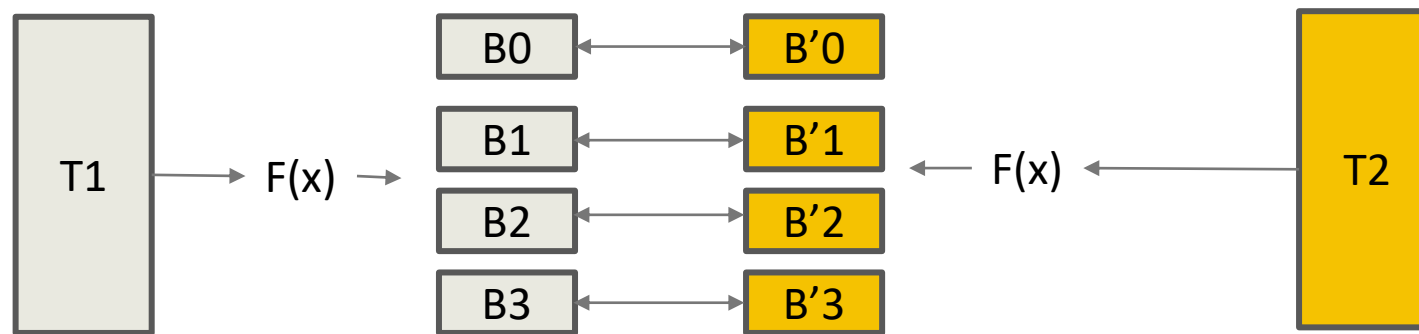
FINSI

FINSI

FIN TANT QUE

PRINCIPE DE LA JOINTURE PAR HACHAGE

1. Hacher la plus petite des deux tables en n fragments
2. Hacher la seconde table, avec la même fonction, en n autres fragments
3. Réunir fragments par paire, et faire la jointure entre les fragments



Exemple : fonction de hachage Modulo 4

COÛT JOINTURE PAR HACHAGE

1. Chargement de T1 : $|T1|$
2. Hachage de T1, création des groupes en RAM : 0
3. Déchargement des groupes en MS : $|T1|$
4. Chargement de T2 : $|T2|$
5. Hachage de T2, création des groupes en RAM : 0
6. Déchargement des groupes en MS : $|T2|$
7. Chargement du Block B_i de T1 avec B'_i de T2, faire la jointure entre les deux blocs : $|B_i| + |B'_i|$
8. Au total : $\sum |B_i| + |B'_i| = |T1| + |T2|$

$$\text{Coût Total} = 3(|T1| + |T2|)$$

BOUCLE INDEXÉE

$T1 \bowtie_b T2$

❑ Comme boucles imbriquées mais avec l'utilisation de l'index défini sur l'attribut b de T2

1. Charger une partie de T1 en RAM
2. Charger l'index
3. Pour chaque tuple de T1, parcourir l'index, retourner les RID des tuples de T2 à joindre
4. Charger ces tuples en RAM
5. Coller les tuples

❑ Coût = $|Index| + |T1| + |T1| * |T2| / NDIST(T2, b)$

COMMENT SE FAIT L'OPTIMISATION

☐ Chercher le meilleur plan d'exécution?

☐ coût excessif

☐ Solution approchée à un coût raisonnable

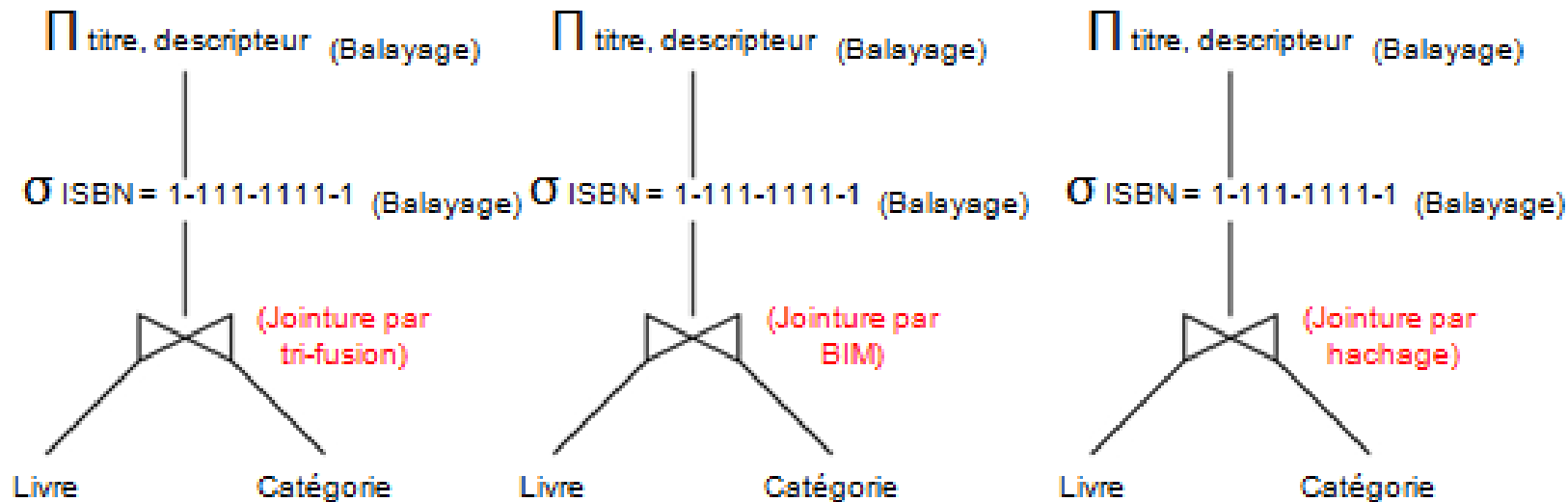
☐ Générer les alternatives

☐ heuristiques

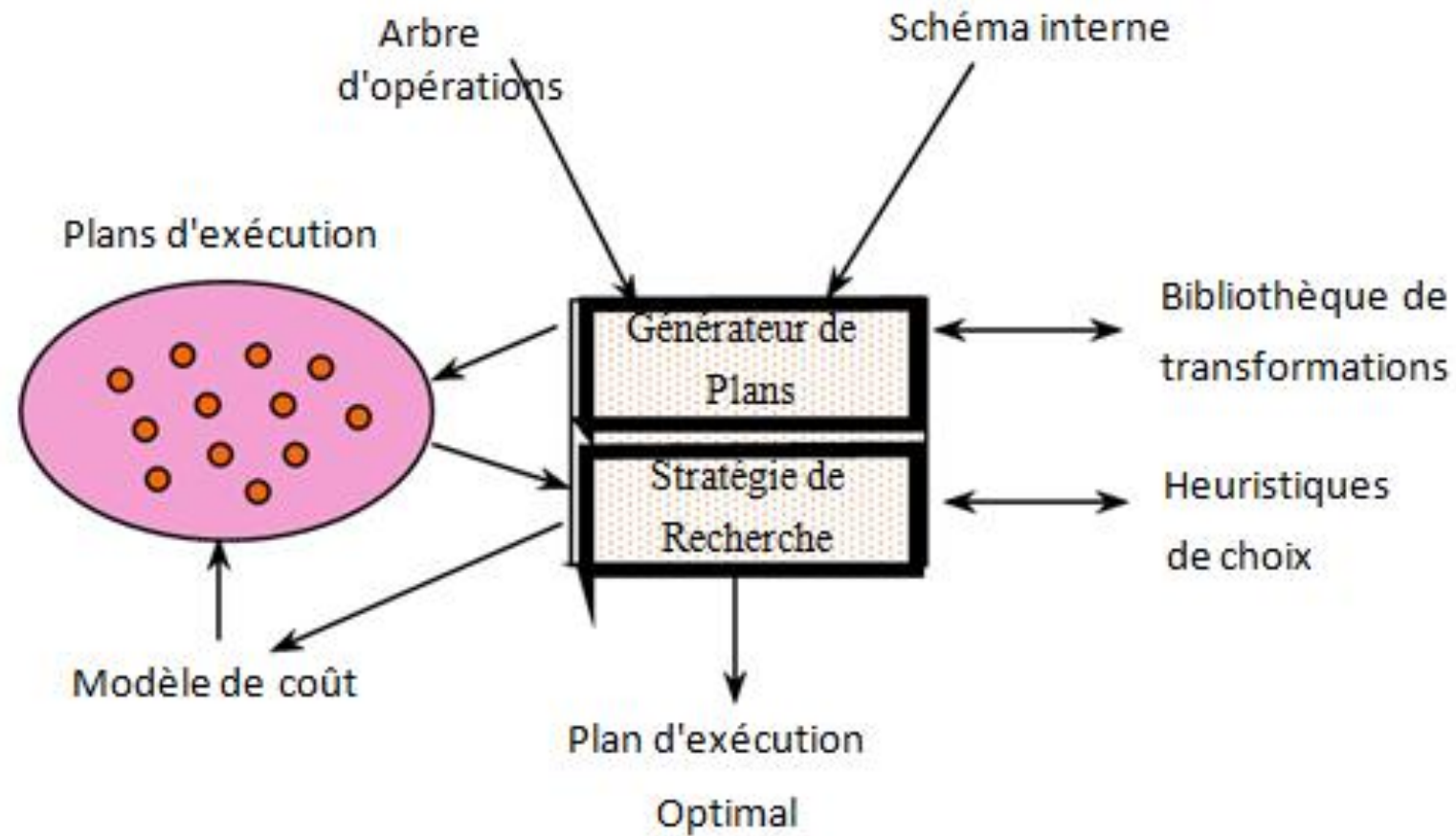
☐ Choisir la meilleure estimation approximative du coût

PLUSIEURS PLANS D'EXÉCUTION POUR UN ARBRE ALGÈBRE

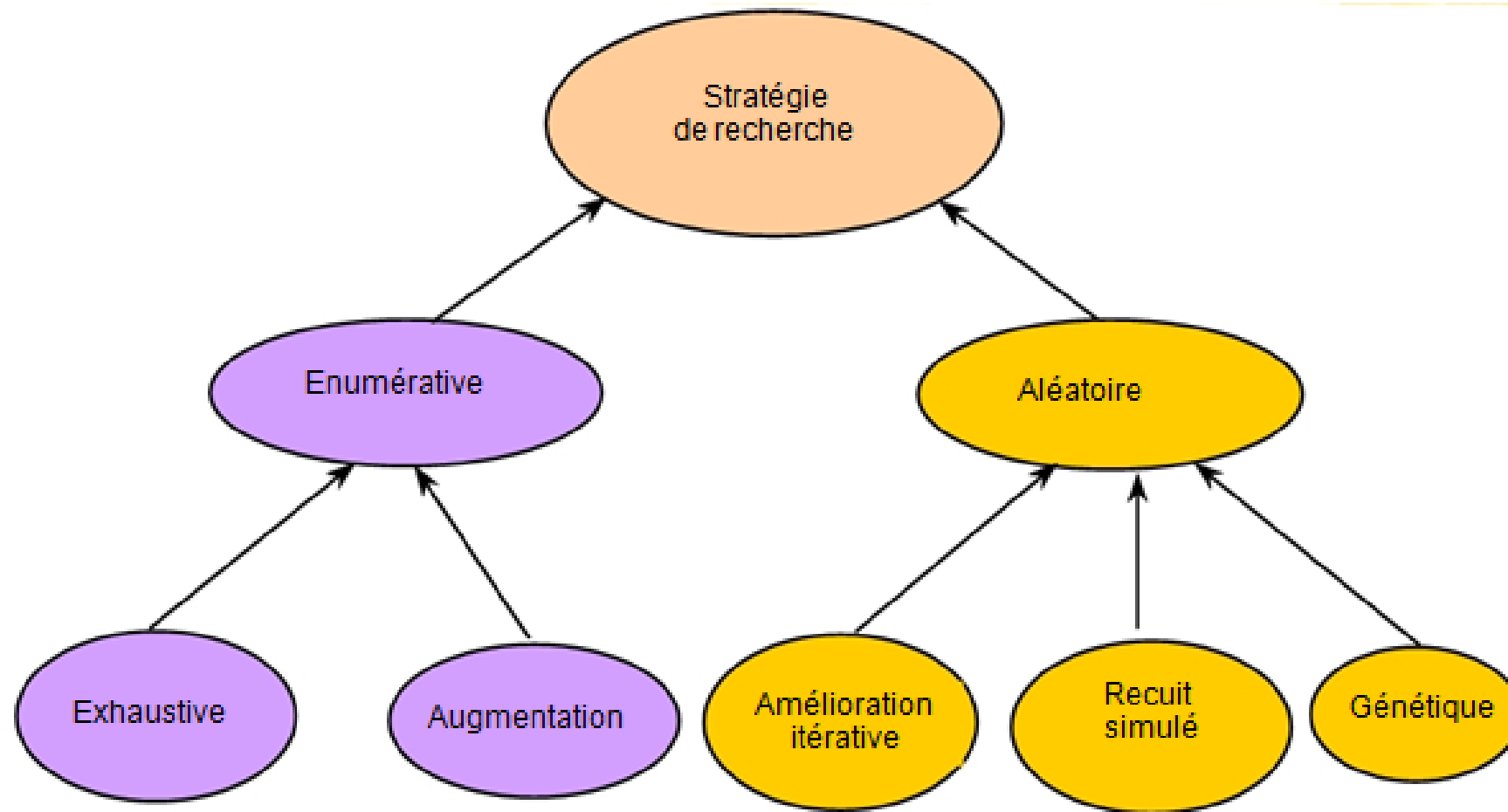
- ❑ Pour chaque opération logique
 - ❑ plusieurs choix d'opérations physiques



CHOIX DU MEILLEUR PLAN



DIFFÉRENTES STRATÉGIES



OPTIMISATION BASÉE SUR MODÈLE DE COÛT

❑ Soit Q une requête à optimiser

❑ Procédure

1. Énumérer tous les plans $\{P_1, \dots, P_m\}$ pour chaque requête (notons que chaque requête possède un ensemble d'opérations $O_1, \dots, O_k$)

2. Pour chaque plan P_j

❑ Pour chaque opération O_i du plan P_j , énumérer les chemins d'accès

❑ Sélectionner le chemin ayant le coût le moins élevé

$$\text{Coût}(P_i) = \sum_{l=1, k} \min(O_l)$$

3. **Coût (Q): $\sum_{h=1, m} \text{Coût}(P_h)$**